

AZ-Delivery

Welcome!

Thank you for purchasing our *AZ-Delivery KY-040 Rotary Encoder Module*.
On the following pages, you will be introduced to how to use and set up this handy device.

Have fun!

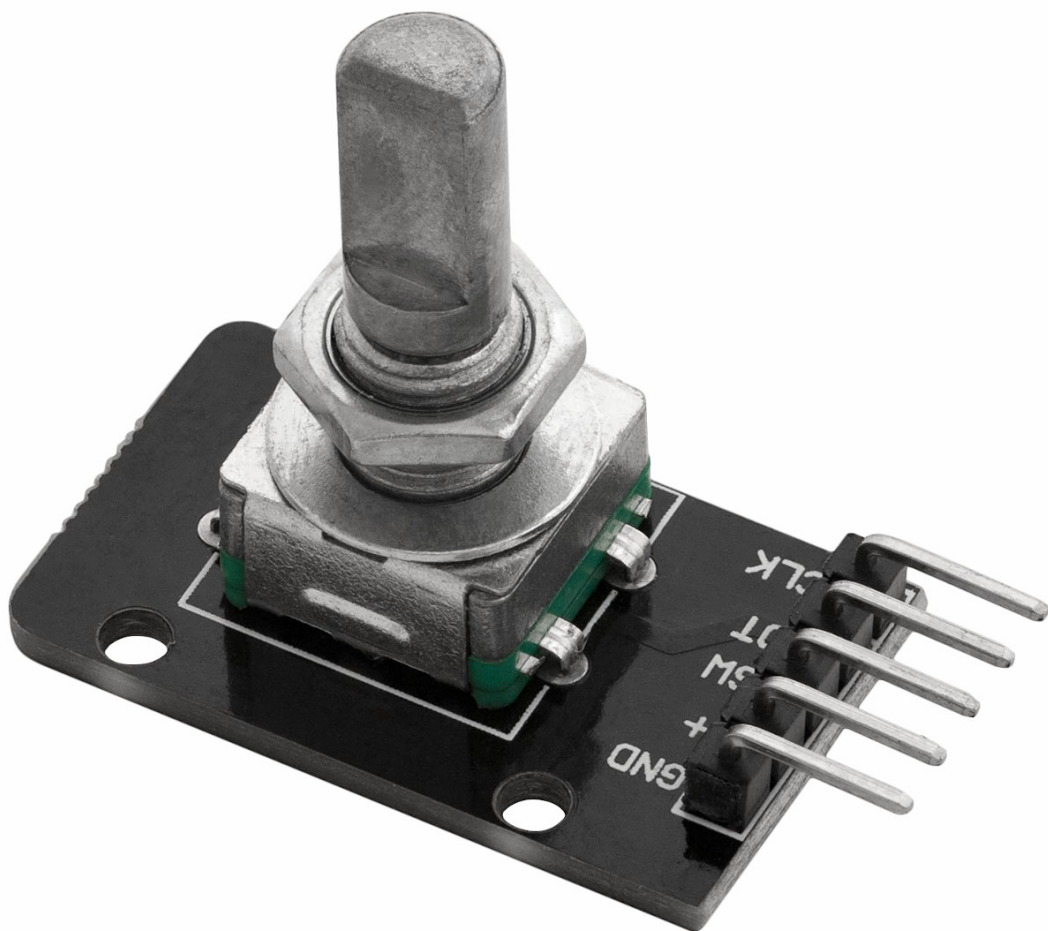




Table of Contents

Introduction.....	3
Working principle.....	4
The output.....	6
Specifications.....	7
The pinout.....	7
How to set-up Arduino IDE.....	8
How to set-up the Raspberry Pi and Python.....	12
Connecting the module with Uno.....	13
Sketch example.....	14
Connecting the module with Raspberry Pi.....	18
Python script.....	19



Introduction

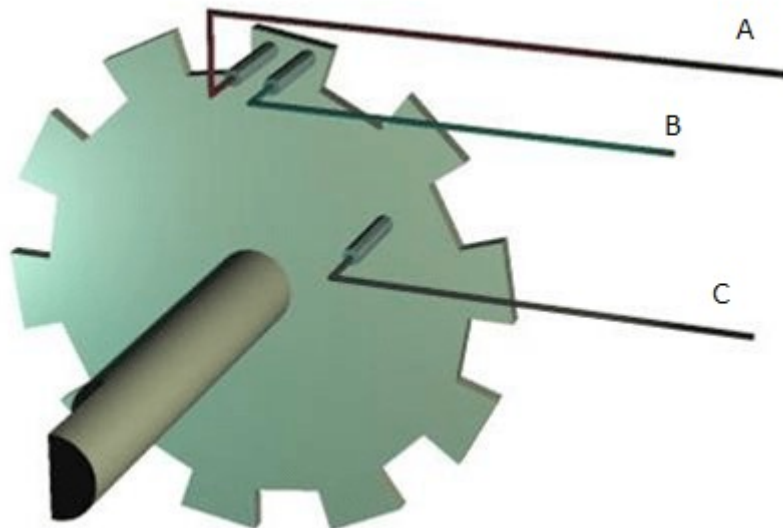
The KY-040 rotary encoder module is a rotary input device (as in knob) that provides an indication of how much the knob has been rotated and in what direction it is rotating. A rotary encoder is an electro-mechanical device that converts rotational motion into digital or analog information. It is a great device for stepper and servo motor control.

A rotary encoder has a fixed number of positions per revolution. These positions are easily felt as small “clicks” when you turn the encoder shaft. The KY-040 rotary encoder module has 20 positions per one full rotation.

Working principle

There are two switches inside the encoder. One switch connects pin A (or CLK) to pin C and the other switch connects pin B (or DT) to C. In each encoder position, both switches are either opened or closed.

The illustration below is the representation of how the switch is constructed.

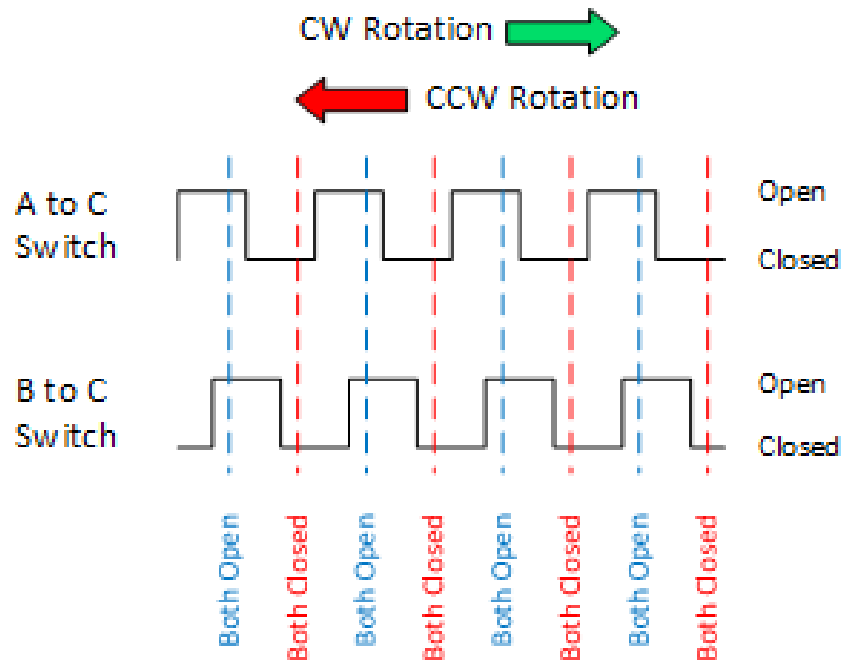


As you can see, the angular position of the A terminal and the B terminal is the following:

- » Rotating the switch clockwise causes the switch connecting A and C to change states first.
- » Rotating the switch counter-clockwise causes the switch connecting B and C to change states first.

Az-Delivery

If the opening and closing of the switches is represented as wave forms, it would look something as shown on the following image:



Essentially, determining which switch changed states first is how the direction of rotation is determined.

- » If A changed states first, the switch is rotating in a clockwise direction.
- » If B changed states first, the switch is rotating in a counter-clockwise direction.



The output

The module is designed so that the *LOW* state is an output when the switches are closed and the *HIGH* state when the switches are open. The *LOW* state is generated by placing a ground at pin *C* and passing it to the *CLK* and *DT* pins when switches are closed. The *HIGH* state is generated connecting the power supply via pull-up resistors to the *CLK* and *DT* pins when switches are open.

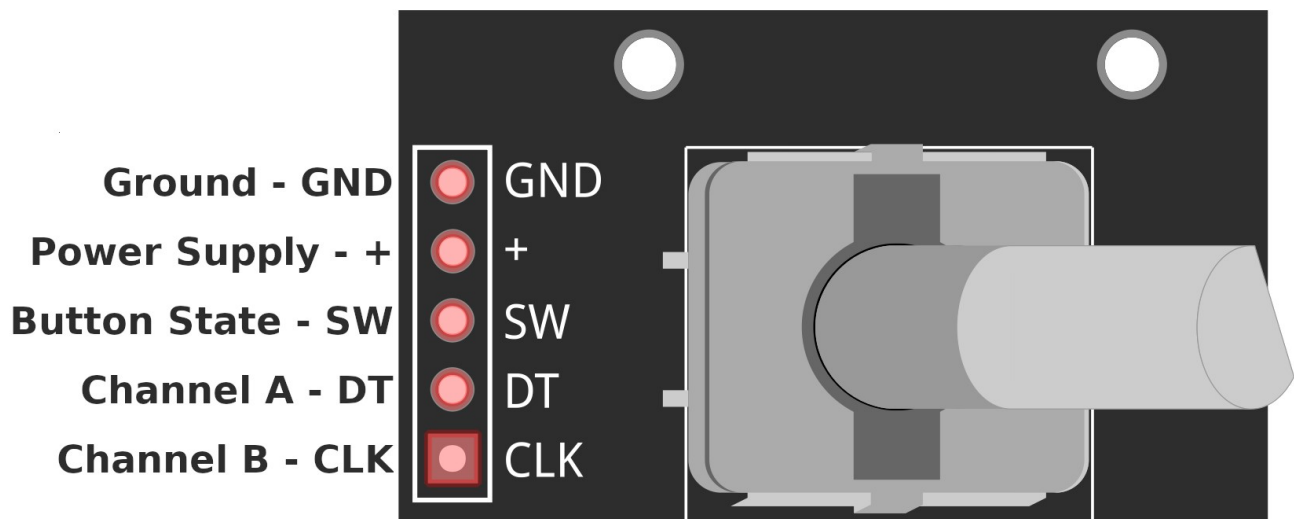
There is a push button that is integral to the encoder. If the shaft is pressed, a normally open switch closes. The *SW* pin outputs the state of this push button.

Specifications

- » Operating voltage range: from 3.3V to 5V DC
- » Operating temperature range: from -40°C to 85°C
- » Output: digital
- » Dimensions: 17 x 29 x 30mm [0.7 x 1.14 x 1.2in]

The pinout

The KY-040 rotary encoder module has five pins. The pinout diagram is shown on the following image:



How to set-up Arduino IDE

If the Arduino IDE is not installed, follow the [link](#) and download the installation file for the operating system of choice.

Download the Arduino IDE



The screenshot shows the Arduino IDE download page. On the left, there is a teal circle containing the Arduino logo (an infinity symbol with a minus sign on the left and a plus sign on the right). To the right of the logo, the text reads: **ARDUINO 1.8.9**
The open-source Arduino Software (IDE) makes it easy to write code and upload it to the board. It runs on Windows, Mac OS X, and Linux. The environment is written in Java and based on Processing and other open-source software.
This software can be used with any Arduino board. Refer to the [Getting Started](#) page for Installation instructions.

On the right side of the page, there are links for different operating systems:

- Windows** Installer, for Windows XP and up
Windows ZIP file for non admin install
- Windows app** Requires Win 8.1 or 10
[Get](#) 
- Mac OS X** 10.8 Mountain Lion or newer
- Linux** 32 bits
- Linux** 64 bits
- Linux** ARM 32 bits
- Linux** ARM 64 bits

At the bottom right, there are links for [Release Notes](#), [Source Code](#), and [Checksums \(sha512\)](#).

For *Windows* users, double click on the downloaded .exe file and follow the instructions in the installation window.

Az-Delivery

For *Linux* users, download a file with the extension *.tar.xz*, which has to be extracted. When it is extracted, go to the extracted directory and open the terminal in that directory. Two *.sh* scripts have to be executed, the first called *arduino-linux-setup.sh* and the second called *install.sh*.

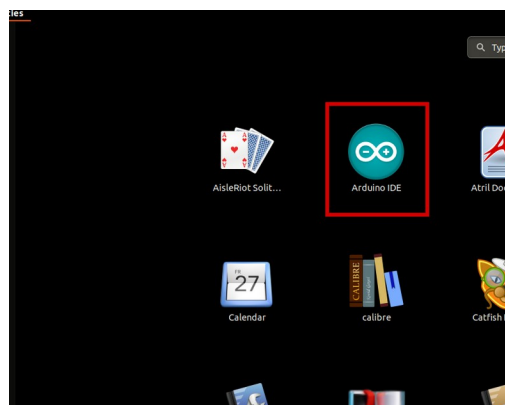
To run the first script in the terminal, open the terminal in the extracted directory and run the following command:

```
sh arduino-linux-setup.sh user_name
```

user_name - is the name of a superuser in the Linux operating system. A password for the superuser has to be entered when the command is started. Wait for a few minutes for the script to complete everything.

The second script called *install.sh* script has to be used after installation of the first script. Run the following command in the terminal (extracted directory): **sh install.sh**

After the installation of these scripts, go to the *All Apps*, where the *Arduino IDE* is installed.



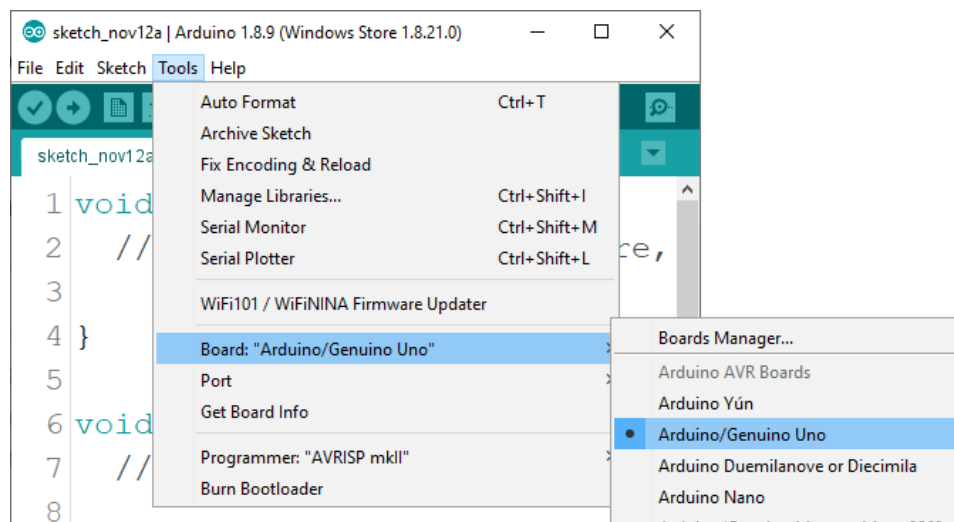
Az-Delivery

Almost all operating systems come with a text editor preinstalled (for example, *Windows* comes with *Notepad*, *Linux Ubuntu* comes with *Gedit*, *Linux Raspbian* comes with *Leafpad*, etc.). All of these text editors are perfectly fine for the purpose of the eBook.

Next thing is to check if your PC can detect an Arduino board. Open freshly installed Arduino IDE, and go to:

Tools > Board > {your board name here}

{your board name here} should be the *Arduino/Genuino Uno*, as it can be seen on the following image:



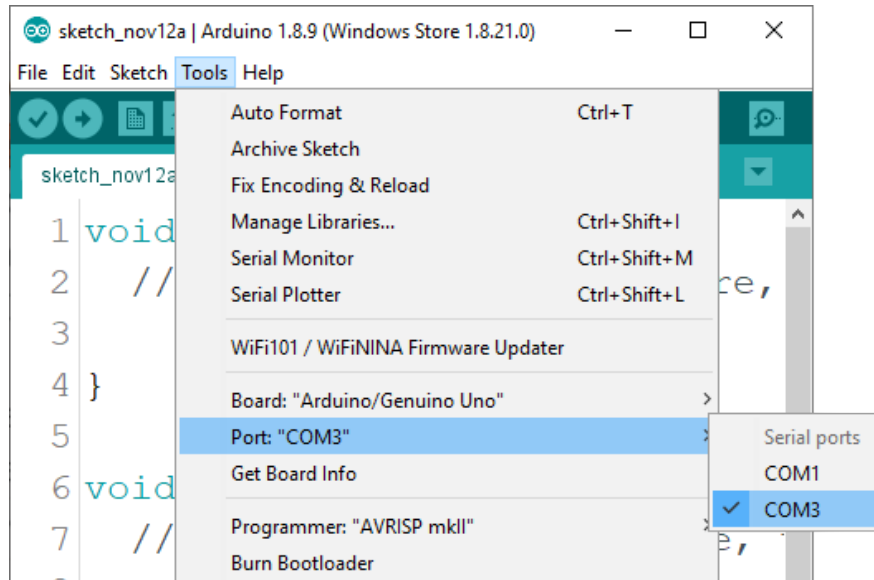
The port to which the Arduino board is connected has to be selected. Go to:

Tools > Port > {port name goes here}

and when the Arduino board is connected to the USB port, the port name can be seen in the drop-down menu on the previous image.

Az-Delivery

If the Arduino IDE is used on Windows, port names are as follows:



For *Linux* users, for example port name is `/dev/ttyUSBx`, where *x* represents integer number between 0 and 9.



How to set-up the Raspberry Pi and Python

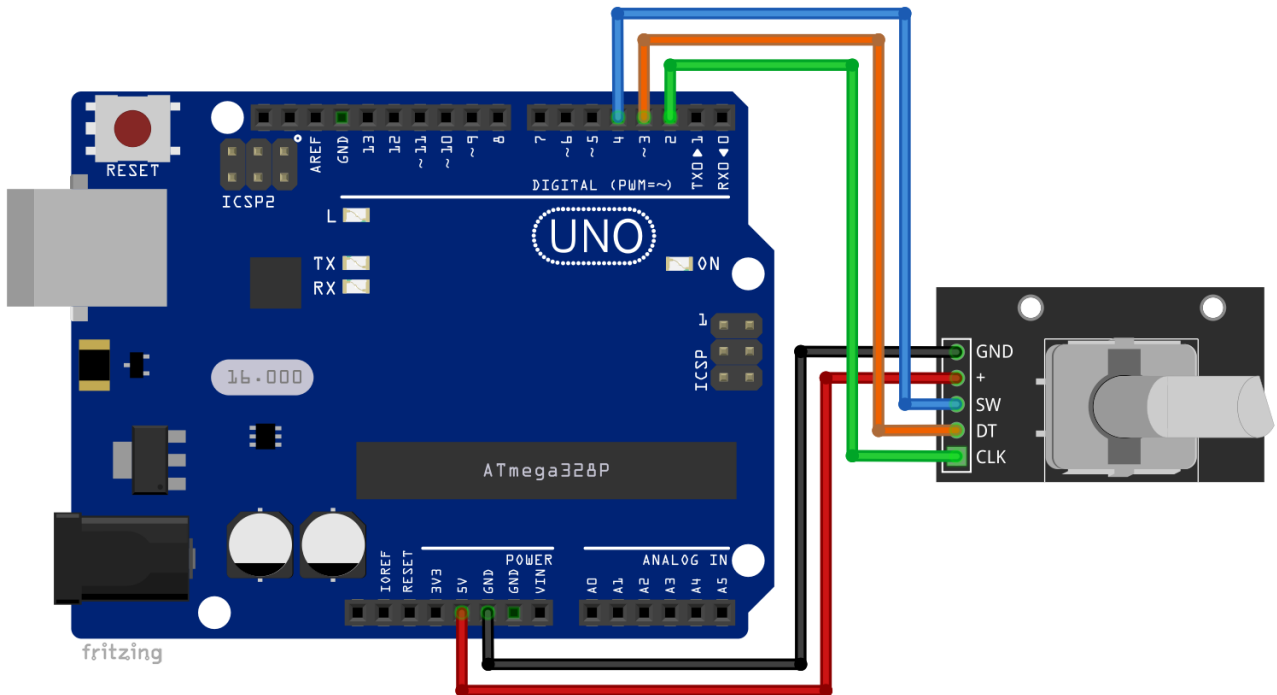
For the Raspberry Pi, first the operating system has to be installed, then everything has to be set-up so that it can be used in the *Headless* mode. The *Headless* mode enables remote connection to the Raspberry Pi, without the need for a *PC* screen Monitor, mouse or keyboard. The only things that are used in this mode are the Raspberry Pi itself, power supply and internet connection. All of this is explained minutely in the free eBook:

[Raspberry Pi Quick Startup Guide](#)

The *Raspbian* operating system comes with *Python* preinstalled.

Connecting the module with Uno

Connect the KY-039 module with the Uno as shown on the following connection diagram:



KY-040 pin	>	Uno pin
+ (VCC)	>	5V
GND	>	GND
CLK	>	D2
DT	>	D3
SW	>	D4

Red wire

Black wire

Green wire

Orange wire

Blue wire

Az-Delivery

Sketch example

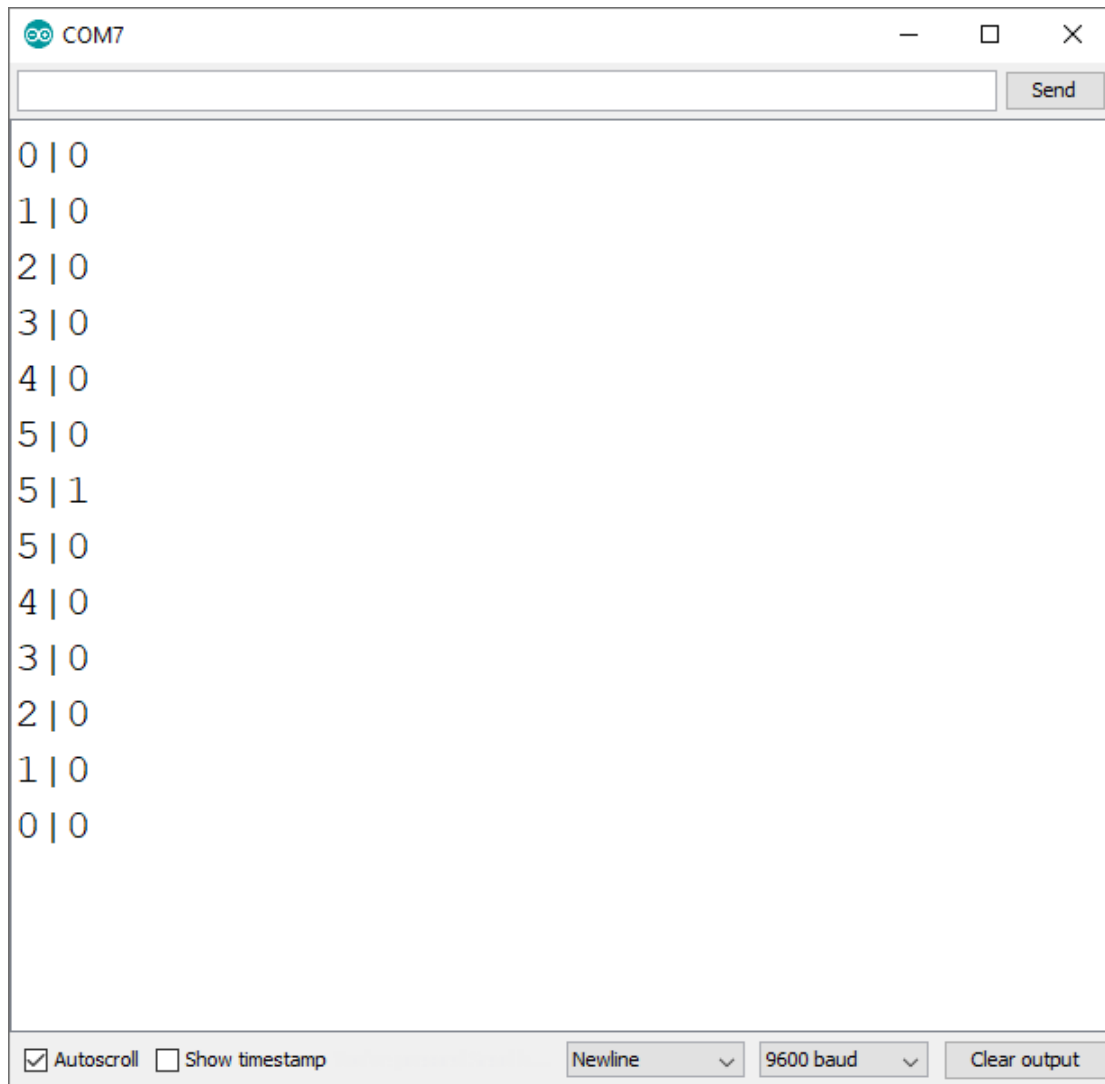
```
#define CLK_PIN 2
#define DT_PIN 3
#define SW_PIN 4
int position = 0;
int last_position = 0;
int n = 0;
bool taster = LOW;
bool last_taster = LOW;
void setup() {
    pinMode(CLK_PIN, INPUT_PULLUP);
    pinMode(DT_PIN, INPUT_PULLUP);
    pinMode(SW_PIN, INPUT_PULLUP);
    Serial.begin(9600);
}
void loop() {
    n = digitalRead(CLK_PIN);
    taster = !digitalRead(SW_PIN);
    if(taster != last_taster) {
        Serial.print(position);
        Serial.print("|");
        Serial.println(taster);
        delay(10);
        last_taster = taster;
    }
}
```

Az-Delivery

```
// one tab
if((last_position == 0) && (n == HIGH)) {
    if(digitalRead(DT_PIN) == LOW) {
        position++;
    }
    else {
        position--;
    }
    Serial.print(position);
    Serial.print("|");
    Serial.println(taster);
}
last_position = n;
}
```

Az-Delivery

Upload the sketch to the Uno and open Serial Monitor (*Tools > Serial Monitor*). The result should look like the output on the following image:



To get these values, move or press the shaft of the rotary encoder.

NOTE: There is a lot of noise in the output of the module. A high number of errors is expected with high noise. How to clear output out of the noise is not in the scope of this eBook.

Az-Delivery

The sketch starts with creating three macros called *CLK_PIN*, *DT_PIN* and *SW_PIN*. These macros represent the digital pins of Uno on which pins of the module are connected.

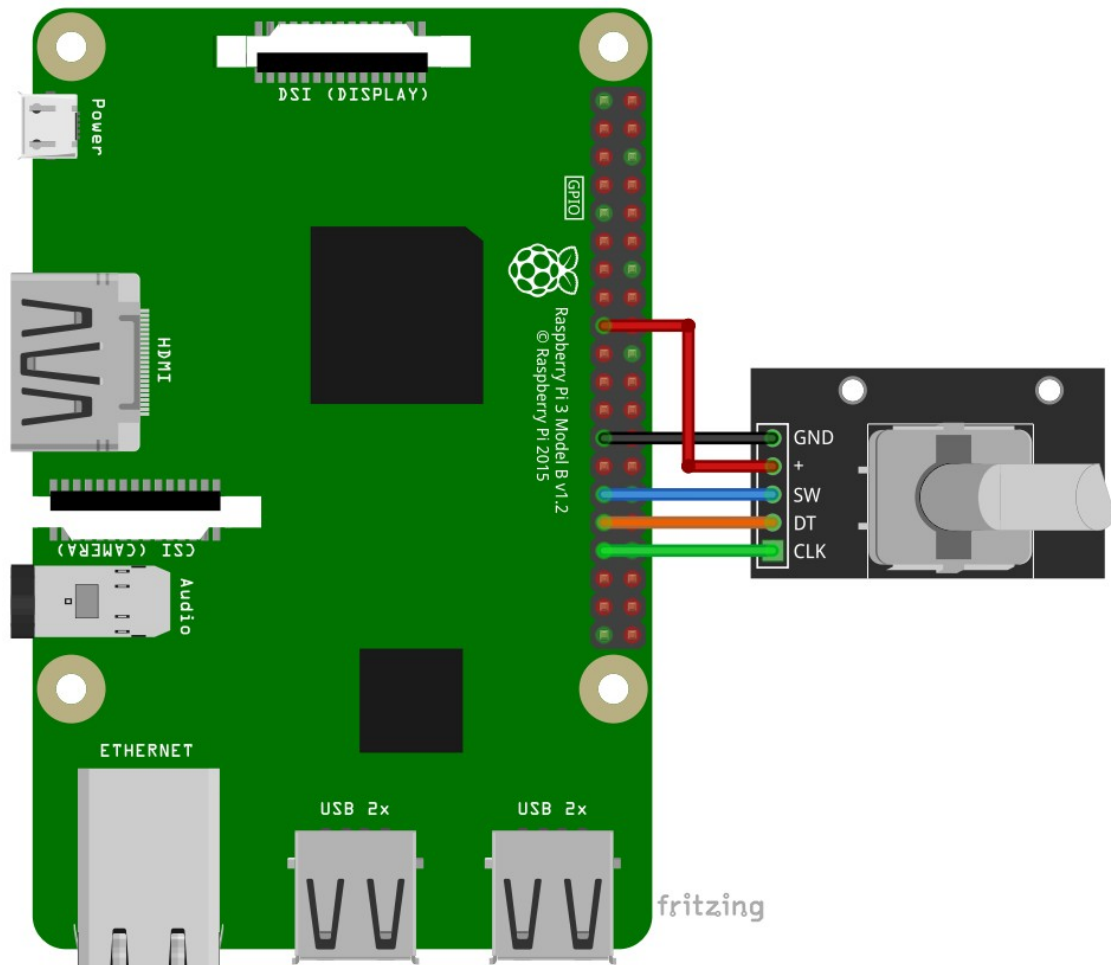
Next, several variables are created which are used in the algorithm that detect when and in which direction the shaft of the encoder is moving. The variable called *position* holds the value which represents the current position of the shaft. At the beginning of the sketch execution this value is zero, but when the shaft of the encoder is moved, it can hold both positive or negative values depending on the direction of moving shaft (positive value - clockwise direction, negative value - counterclockwise direction).

In the *setup()* function the pin modes for all three pins are set to *INPUT_PULLUP*. Next, the serial communication is started with a baud rate of *9600bps*.

In the *loop()* function the state of the module pins is read. Then, the algorithm for detecting movement of the shaft is executed, checking the previously read states of the pins. This algorithm is not covered in this eBook. The output of this algorithm is a message displayed in the Serial Monitor (image of Serial Monitor on the previous page).

Connecting the module with Raspberry Pi

Connect the module with the Raspberry Pi as shown on the following connection diagram:



KY-040 pin	>	Raspberry Pi pin	
+ (VCC)	>	3V3 [pin 17]	Red wire
GND	>	GND [pin 25]	Black wire
SW	>	GPIO5 [pin 29]	Orange wire
DT	>	GPIO6 [pin 31]	Blue wire
CLK	>	GPIO13 [pin 33]	Green wire



Python script

```
import RPi.GPIO as GPIO
from time import sleep

GPIO.setmode(GPIO.BCM)
GPIO.setwarnings(False)

SW_PIN = 5
DT_PIN = 6
CLK_PIN = 13

GPIO.setup(CLK_PIN, GPIO.IN)
GPIO.setup(DT_PIN, GPIO.IN)
GPIO.setup(SW_PIN, GPIO.IN, pull_up_down=GPIO.PUD_UP)

position = 0

a = 0
b = 0

a_last = 1
b_last = 1

taster = False
last_taster = False
```

Az-Delivery

```
print('[Press CTRL + C to end the script!]\n')
try: # Main program loop
    while True:
        a, b = GPIO.input(CLK_PIN), GPIO.input(DT_PIN)
        taster = not GPIO.input(SW_PIN)
        if taster != last_taster:
            print('{}|{}'.format(position, taster))
            sleep(0.01)
            last_taster = taster
        if a != a_last or b != b_last:
            if a == 0 and b == 1:
                while not (a == 1 and b == 1):
                    a = GPIO.input(CLK_PIN)
                    b = GPIO.input(DT_PIN)
                position += 1
                print('{}|{}'.format(position, taster))
            if a == 1 and b == 0:
                while not (a == 1 and b == 1):
                    a = GPIO.input(CLK_PIN)
                    b = GPIO.input(DT_PIN)
                position -= 1
                print('{}|{}'.format(position, taster))
            a_last = a
            b_last = b

except KeyboardInterrupt:
    print('\nScript end!\n')

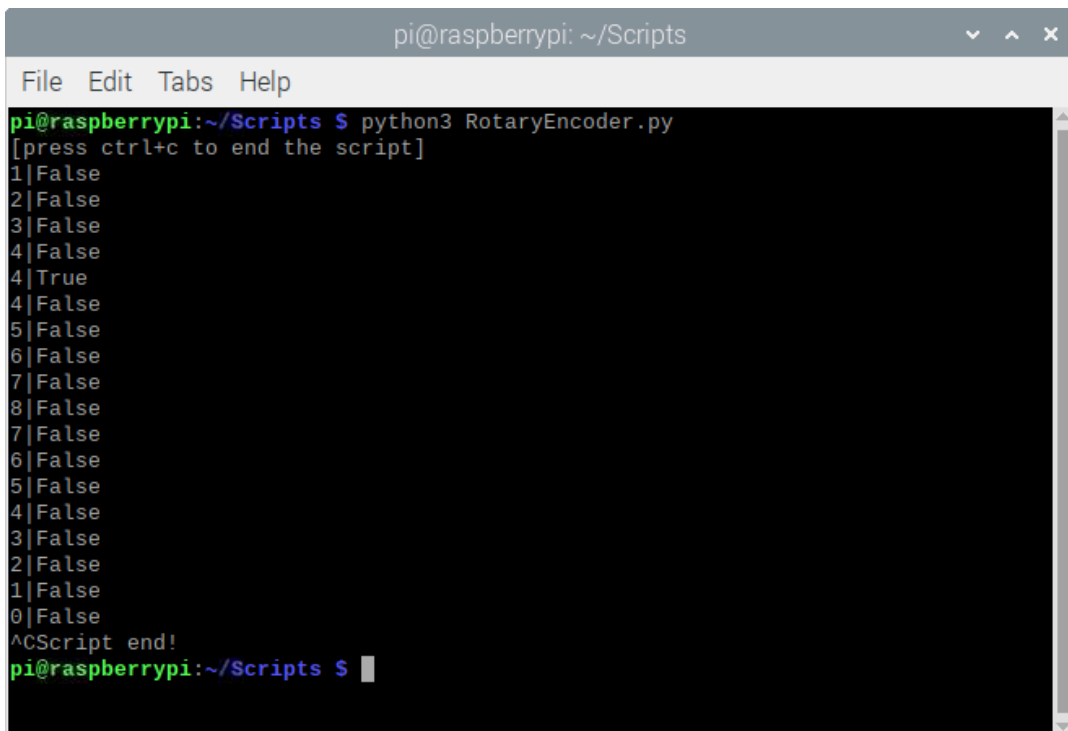
finally:
    GPIO.cleanup()
```

Az-Delivery

Save the script by the name *RotaryEncoder.py*. To run the script, open the terminal in the directory where the script is saved and run the following command:

python3 RotaryEncoder.py

The result should look like the output on the following image:



```
pi@raspberrypi: ~/Scripts
File Edit Tabs Help
pi@raspberrypi:~/Scripts $ python3 RotaryEncoder.py
[press ctrl+c to end the script]
1|False
2|False
3|False
4|False
4|True
4|False
5|False
6|False
7|False
8|False
7|False
6|False
5|False
4|False
3|False
2|False
1|False
0|False
^CScript end!
pi@raspberrypi:~/Scripts $
```

To stop the script press *CTRL* + *C* on the keyboard. To get these values, move or press the shaft of the rotary encoder.

The script does the same thing on the same way as the sketch for the Uno, the only difference is that the script is written in *Python*.

NOTE: There is a lot of noise in the output of the module. A high number of errors is expected with high noise. How to clear output out of the noise is not in the scope of this eBook.

Az-Delivery

Now it is the time to learn and make your own projects. You can do that with the help of many example scripts and other tutorials, which can be found on the internet.

If you are looking for the high quality products for Arduino and Raspberry Pi, AZ-Delivery Vertriebs GmbH is the right company to get them from. You will be provided with numerous application examples, full installation guides, eBooks, libraries and assistance from our technical experts.

<https://az-delivery.de>

Have Fun!

Impressum

<https://az-delivery.de/pages/about-us>